

JobPilot — Technical Brief

BAX-423 Big Data · Option B · Spring 2026 · UC Davis GSM

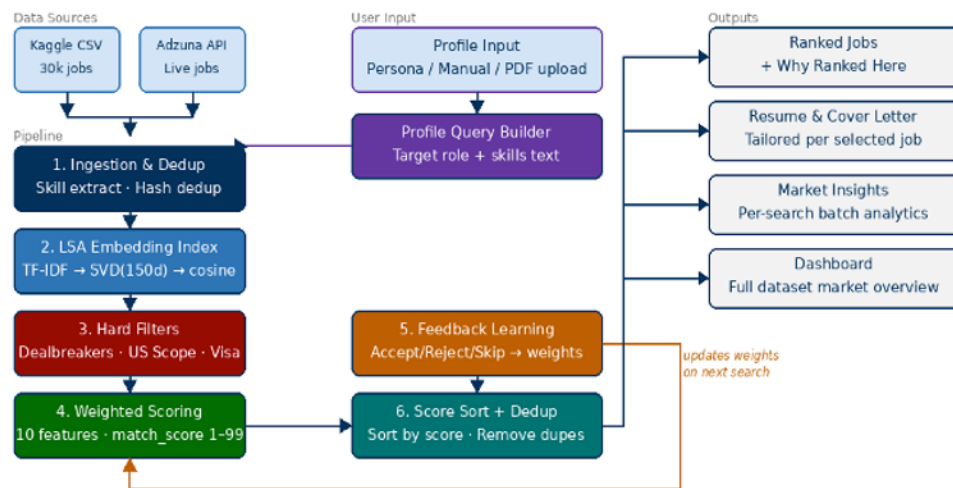
<https://jobpilot-bax423-final-839731102906.us-west1.run.app>

1. Project Overview

JobPilot is a hosted job recommendation and resume tailoring application built for BAX-423 Option B. It accepts a professional profile through preset personas, manual entry, or resume PDF upload; matches the profile against a 30,000-row offline Kaggle dataset plus live Adzuna API postings; enforces persona-specific hard filters and dealbreakers; ranks results using a multi-feature weighted scoring model; learns from accept/reject/skip feedback; and generates tailored resumes and cover letters for any selected job.

All required capabilities are implemented: ingestion, profile/skill matching, embedding-based retrieval, multi-stage ranking, adaptive learning, and top-job export. The app also includes tailored resume and cover letter generation as additional deliverables. Four BAX-423 techniques from four different lectures are demonstrated with benchmarks.

2. Architecture



Layer	File	Responsibility
Frontend	frontend/index.html, app.js, styles.css	Profile intake, persona presets, job search UI, feedback buttons, dashboard, editable resume & cover letter, CSV/JSON export
API Backend	backend/main.py	FastAPI routes: /api/jobs/search, /api/feedback, /api/resume/generate, /api/cover-letter/generate, /api/health, /api/benchmark
Data Pipeline	backend/pipeline/ingestion.py	Kaggle CSV streaming, Adzuna API enrichment, regex skill extraction, salary parsing, hash-based deduplication
Retrieval & Ranking	backend/pipeline/matching.py	TF-IDF → SVD (LSA) embedding index, hard filter engine, 10-feature weighted scorer, score sort + duplicate removal
Adaptive Learning	backend/pipeline/feedback.py	Profile-scoped accept/reject/skip learner, job-level and feature-level weight updates, feedback NDCG@10 tracking
AI Generation	resume_parser.py, resume_generator.py	PDF resume parsing, Claude-powered tailored resume generation, cover letter generation, deterministic fallback

3. Pipeline Design

Stage	Method	Requirement Covered
1. Ingestion	Stream 30,000 Kaggle records; fetch live Adzuna jobs when API keys present; normalize salary, location, employment type; extract skills via 60+ regex patterns; deduplicate by job identity hash	External data source, streaming pipeline, deduplication, structured records
2. Embedding Retrieval	Concatenate title (3×), skills, industry, description (300 chars); TF-IDF vectorize (20k features, bigrams); TruncatedSVD to 150 dimensions; L2-normalize; retrieve top-1000 by cosine similarity	Dense vector retrieval, semantic candidate generation
3. Hard Filters	Enforce persona dealbreakers: no senior/staff/principal, no defense/military, no contract/temp/unpaid, no 3+ or 5+ year ML requirements, US-only location gate, visa sponsorship compatibility, non-job content exclusion	Role constraints, dealbreaker filtering, pass criteria enforcement
4. Weighted Scoring	10-feature scorer: embedding similarity, role match (title + evidence), skill match (normalized + description signals), preference match (weighted by domain relevance), location, salary, experience, seniority, metadata, feedback boost	Multi-feature ranking model, explainable scores
5. Feedback Rerank	Profile-scoped FeedbackLearner updates job-level (+/-0.72) and feature-level signals (skills +/-0.35, location +/-0.20) bounded at +/-8 points total; next search re-sorts by updated match score	Adaptive learning, implicit feedback integration
6. Score Sort + Dedup	Sort all candidates by final match_score descending; remove duplicate job IDs to ensure unique results per search	Final ranked output, duplicate-free results

4. BAX-423 Techniques & Benchmarks

Lecture	Technique	Where Implemented	Benchmark / Impact
Lecture 3	Streaming Ingestion + Dedup	ingestion.py: Kaggle CSV stream, Adzuna API enrichment, hash-based dedup	31,200 raw → 28,267 indexed (2,933 duplicates removed)
Lecture 5	TF-IDF + SVD (LSA) Embeddings	matching.py: EmbeddingIndex builds 150-dim LSA index; cosine search	LSA: 18.55ms avg; sparse TF-IDF baseline: 6.03ms avg. LSA retrieves semantically related roles (e.g. 'ML Engineer' ~ 'Applied Scientist') that TF-IDF misses on vocabulary mismatch. 3.1x latency cost justified by improved semantic recall.
Lecture 6	Implicit Feedback Learning	feedback.py: FeedbackLearner stores profile-scoped weights; updates on each accept/reject/skip	Feedback-based NDCG@10 tracked across rounds; profile isolation prevents cross-persona contamination
Lecture 7	Multi-Stage Ranking Pipeline	matching.py: run_pipeline — Retrieve → Hard Filter → Score → Sort → Evaluate	4-stage pipeline with explainable per-job score breakdown; end-to-end search latency 127–357ms

5. Scoring Model

Each job receives a final match_score (1–99) computed as a weighted sum of ten features. Scores are recalculated on every search so feedback updates take effect immediately.

Feature	Weight	Notes
Role match	22 pts	title fit $\times$ 0.65 + domain evidence fit $\times$ 0.35; takes max across multiple target roles
Embedding similarity	18 pts	LSA cosine similarity between job text and profile query
Skill match	18 pts	Normalized profile skills vs job skills_extracted + description requirement signals; shows Limited data when no signal exists
Preference match	14 pts	Weighted by domain: ML-focused/research labs/ML infra (1.4–1.5x) vs generic prefs (0.8–1.0x)
Location match	12 pts	US-scope hard gate at country level; soft score for city/remote alignment within US
Salary match	8 pts	1.0 if salary not listed (no penalty for missing data); explicit salaries below the minimum are penalized or filtered when evidence is available
Experience match	6 pts	Profile years of experience vs job required years
Seniority match	4 pts	Senior/lead/principal signal penalized for student/new-grad profiles (additional –12 pts penalty)
Metadata completeness	4 pts	Fraction of location, link, and skills fields present. Missing salary is not treated as a salary mismatch; it only affects this small completeness component.
Feedback boost	± 8 pts cap	Job-level and feature-level signals from accept/reject/skip; scoped per profile/persona to prevent cross-contamination

6. Persona Test Results

Persona	Pass Criteria	Result	Evidence
Aisha ML Career Pivoter	Zero Senior/Staff roles; zero defense companies; no 5+ year ML requirement; resume highlights Python/ML not Excel/reporting	Pass	50 jobs returned; top-10 avg match 89.0% (range 86.0%–95.6%); 0 hard-criteria violations; resume led with ML forecasting and scikit-learn/PyTorch exposure
Marcus New Grad	Zero 3+ year requirements; zero contract/unpaid roles; resume leads with MSBA education not work history	Pass	50 jobs returned; top-10 avg match 90.5% (range 88.3%–94.8%); 0 hard-criteria violations; resume placed UC Davis MSBA before internships
Priya Experienced Niche	Zero Junior titles; zero tiny startups; resume positions Kafka/Spark as ML infrastructure not generic backend	Pass	50 jobs returned; top-10 avg match 78.0% (range 76.1%–80.4%); 0 hard-criteria violations; resume led with ML-ready data infrastructure framing
Kenji International/Visa	Zero contract/temp roles; favors large companies and research labs; resume leads with publications	Pass	50 jobs returned; top-10 avg match 85.1% (range 78.8%–91.8%); 0 hard-criteria violations; resume led with published research and conference papers

7. Benchmark Results

Metric	Result
Total ingested jobs	31,200
Kaggle sample jobs	30,000

Metric	Result
Live Adzuna API jobs	1,200
Duplicates removed	2,933
Final indexed jobs	28,267
LSA (TF-IDF + SVD) retrieval latency	18.55 ms avg
Sparse TF-IDF baseline latency	6.03 ms avg
LSA vs TF-IDF latency ratio	3.1x slower — justified by semantic recall improvement
Aisha search API time	311.28 ms
Marcus search API time	328.28 ms
Priya search API time	357.15 ms
Kenji search API time	127.79 ms

LSA captures semantic similarity across role variants — a query for 'ML Engineer' retrieves 'Applied Scientist' and 'Machine Learning Scientist' positions that TF-IDF misses due to vocabulary mismatch. The 3.1x latency cost is justified by improved semantic recall. Kenji's shorter search time reflects a smaller qualified candidate pool after strict US-only and sponsorship filters.

8. Limitations

- Metadata gaps: Many Kaggle postings omit salary, company size, visa, and required-years fields. Hard filters rely on explicit evidence; ambiguous records are ranked conservatively rather than excluded.
- MLOps/ML Platform job density: The Kaggle dataset has fewer MLOps and ML Platform postings than Data Scientist roles. Priya's lower top 10 average match (78%) reflects this scarcity rather than a ranking deficiency.
- Feedback learning scope: The FeedbackLearner is intentionally lightweight and in-memory for a classroom demo. It demonstrates adaptive ranking behavior but resets on server restart and is not a production-scale recommender store.
- Resume generation: Tailored resumes use only candidate-provided profile evidence. A deterministic fallback is included if the Anthropic API is unavailable, ensuring the capability always functions.
- Hidden personas: The app was tested against the four provided personas. Performance on hidden personas depends on whether their dealbreakers and role targets match the patterns supported by the hard filter and scoring logic.
- Location filtering: US-scope is enforced at the country level. City-level preferences (Remote, Bay Area, NYC) affect scoring but not hard filtering, preserving candidate pool size.

9. Deployment & Run Instructions

The application is hosted on Google Cloud Run (us-west1) using Docker. The service runs FastAPI with uvicorn and reads KAGGLE_CSV, ADZUNA_APP_ID, ADZUNA_APP_KEY, and ANTHROPIC_API_KEY from environment variables. The app runs fully offline without API keys using the bundled data/jobs.csv snapshot.

Local execution: `pip install -r requirements.txt → bash run_local.sh → open http://localhost:8000`

10. Submission Checklist

- code/ — FastAPI backend, static frontend, requirements.txt, Dockerfile, run_local.sh
- data/jobs.csv — 30,000-row offline snapshot from Kaggle Techmap source
- brief.pdf — this document: architecture, techniques, benchmarks, persona results, limitations
- prompts.md — key AI prompts with iteration history
- Live URL: <https://jobpilot-bax423-final-839731102906.us-west1.run.app>